EN.705.744.SP26 - Continuum Memory Systems: Multi-Frequency Memory Blocks for Reduced Forgetting in Transformers

T. Hendricks

May 2026

1 Abstract

Continual learning in large language models remains a significant challenge due to catastrophic forgetting. While there are approaches which exist to adjust model knowledge, they do not provide robust mechanisms for long-term memory formation without degrading prior knowledge. This work investigates the Continuum Memory System (CMS), a transformer architectural modification proposed in a 2025 Google paper, which replaces standard feedforward network (FFN) blocks with multi-frequency memory blocks (CMS blocks) within a transformer to better regulate parameter updates over time such that models can have better long term memory retention [1]. To evaluate CMS, DistilBERT was modified by replacing its FFN layers with sequential CMS blocks and tested in a domain-incremental continual learning setting using the CLINC150 dataset. Performance was evaluated using a per-domain accuracy heat-map matrix, calculating the Average Diagonal Accuracy (ADA), Average Accuracy (ACC), and Backward Transfer (BWT). Results showed that while CMS implementation did not eliminate catastrophic forgetting in the encoder-only model, it did have reduced forgetting of previously learned tasks, as indicated by improved BWT scores relative to the baseline model. This improvement was noticed to come with reduced immediate learning performance, reflected in lower ADA and ACC scores. These findings suggest that while CMS is a promising direction for improving retention in transformer-based models, this architectural change on its own is not enough to reduce catastrophic forgetting by a significant margin and may benefit from further discovery in balancing memory retention with learning capacity.

2 Introduction

Despite the growing social phenomena of large language model (LLM) usage as a daily tool in modern society, there is a major shortcoming: LLMs are mostly static. While there are techniques in place to change the behavior (such as fine-tuning) or increase knowledge (such as in-context learning or retrieval augmented generation) of a model, continual learning after initial training proves challenging with side effects of catastrophic forgetting or reaching context-length limits. Google Researchers, in their 2025 paper "Nested Learning: The Illusion of Deep Learning Architecture", hereon may be referred to as NL paper, portrayed LLMs akin to individuals with anterograde amnesia due to the fact that modern LLMs cannot form new long-term memories. Rather, they are confined to (1) the present so long as it fits within their context window and (2) long-term past knowledge accumulated through their original training [1]. The question arises, what are the current techniques that do not impact context length that LLMs can use to form new long term memories? And while we could get into the various techniques and their differences (i.e fine-tuning, online learning, reinforcement learning, etc.), they all come down to the same fundamental principle, the answer to the question: model weight updates. Updating the weight of the model will never

run into the issue of reaching context length window limits. However, as the weights are updated, the model runs the risk of progressively losing what it previously learned. This is most commonly referred to as catastrophic forgetting. This, combined with the context window limitation, makes it clear that modern LLMs are not truly able to be high functioning continual learners. And if the goal is for artificial intelligence to mimic the capability of the human brain, then it is in need of some change such that it better mimics neuroplasticity and long-term memory systems.

This issue is the catalyst for Google's 2025 NL paper. In that, researchers claimed that continual learning and long-context reasoning can be enhanced through three contributions: (1) more expressive optimizers, (2) a self-modifying sequence model that learns its own update algorithm, and (3) an architectural adjustment to MLP blocks such that the traditionally uniform structure becomes more like a chain of blocks operating across a spectrum of update frequencies (coined Continuum Memory System (CMS)) [1].

To study this, this research paper focused on just one of the NL paper's findings: the implementation of Continuum Memory System blocks instead of the standard MLP blocks to improve the model's continual learning ability (reduce catastrophic forgetting). To prove this claim, the architecture of DistilBert, an encoder only model, was modified such that its feedforward network (FFN) was replaced with CMS. A continual learning experiment was then built out where the model was trained sequentially on one domain (each domain contained multiple intent categories) at a time, as opposed to joint training where the model sees all domains simultaneously. DistilBert with no changes served as the baseline model. The accuracy of the test data on the current domain and the previously learned domains were compared as a way to visualize catastrophic forgetting. Additionally, Backward Transfer (BWT) was calculated as a more quantifiable measurement of continued learning. The hypothesis was that the base model would present catastrophic forgetting (lower accuracy on previously learned classifications and lower BWT) and the modified version of the model with CMS would do better at retaining pre-learned knowledge while learning and retaining more in the present (higher accuracy and higher BWT).

The experiments demonstrated that replacing DistilBERT's FFN blocks with CMS blocks consistently, while not significantly, reduced catastrophic forgetting relative to the base model. However, a tradeoff was observed: all experiments with CMS configurations showed a reduction in learning capacity on the current task (decreased accuracy), suggesting that the frequency-based update mechanism trades some immediate learning ability for better retention of prior knowledge. These findings indicate that the CMS architecture is a promising direction for reducing catastrophic forgetting in encoder-only transformer models applied to sequential classification tasks but more investigation is needed to yield better results.

3 Background

3.1 Transformer Block: Continuum Memory System

Following the CMS formalization as derived in the NL paper, the below formulas are needed to understand the makeup of the CMS structure [1].

Let f_ℓ be representative of frequency, which is here defined as the number of updates per unit of time.

Given the input:

$$X = \{x_i\}_{i=1}^{T} \tag{1}$$

The output of the CMS chain (forward pass) can be formally written as:

$$y_t = \text{MLP}^{(f_k)}\left(\text{MLP}^{(f_{k-1})}\left(\cdots \text{MLP}^{(f_1)}\left(\mathbf{x}_t\right)\right)\right) \tag{2}$$

Where parameter updates happen every $C^{(\ell)}$ steps:

$$C^{(\ell)} := \frac{\max_i C^{(i)}}{f_\ell} \tag{3}$$

The rule for parameter updating is then:

$$\theta_{i+1}^{(f_\ell)} = \theta_i^{(f_\ell)} - \begin{cases} \sum_{t=i-C^{(\ell)}}^{i} \eta_t^{(\ell)} f(\theta_t^{(f_\ell)}; x_t) & \text{if } i \equiv 0 \pmod{C^{(\ell)}} \\ 0 & \text{otherwise} \end{cases} \tag{4}$$

where:

- $\theta^{(f_\ell)}$ is the parameters of the MLP block associated with frequency f_ℓ that get updated,
- $\eta_t^{(\ell)}$ is the learning rate for block ℓ at step t,
- $f(\theta_t^{(f_\ell)}; x_t)$ is the error component of an arbitrary optimizer.

This rule shows to accumulate the optimizer's error signal over the last $C^{(\ell)}$ steps, scale by the learning rate, and subtract from the current parameters. But only do this every $C^{(\ell)}$ steps, otherwise leave the parameters unchanged.

Applying this idea of parameter updates being controlled by time to the the traditional transformer model training, it can be inferred that maximum frequency and minimum steps are applied since the gradient is applied to adjust the model weights at each step, thereby updating parameters after each FFN (full fine-tuning). In the context of a continual learning environment, this is problematic as it results in the model experiencing catastrophic forgetting due to the constant parameter updates overwriting what it previously learned. Following the parameter update rule (equation 4) proposed in Google's NL paper, the CMS blocks can reduce this loss in knowledge as its MLP blocks update with a spectrum of frequencies, allowing the model at varying rates to update knowledge stored while also recalling past information it has learned. It is important for frequency to vary between MLP blocks as this is what differentiates it from the standard single MLP block typical transformer architecture. If frequency was to remain fixed in the CMS block, then the nested MLP structure would provide little improvement as it would have linear updates which would essentially collapse CMS back into FFN. The differentiation between frequencies is what creates the long-term/short-term memory distinction to assist in remedying catastrophic forgetting.

3.1.1 Architectural Structure of CMS Blocks

There are three variants of the CMS block architecture: nested, sequential, and independent [1]. These vary in their weight initialization method and the specifics as to how the MLP blocks are connected. (1) In the nested variant, the initial state of each MLP block at level $s+1$ is meta-learned from the context of level s. At the end of each chunk window, the block's weights are reset back to this meta-learned initialization, meaning knowledge transfers between levels through the learned starting point rather than through the final parameter state [1]. (2) For the sequential variant, all MLP blocks share a common initialization derived through a one-time meta-learning process anchored to the lowest frequency level's context. Unlike the nested variant, weights are never reset

between chunks, rather the meta-learned initialization serves as the starting point before learning begins, after which weights evolve continuously with the output of block s feeding into block $s+1$ [1]. (3) Lastly, in the independent variant, blocks share the same initialization strategy as sequential CMS but operate in parallel rather than in series. Each block receives the same input independently and their outputs are combined via an aggregation function such as a learnable weighted sum [1].

However, these meta-learned initializations for parameter weights can be simplified by instead using the weights from the FFN of the existing pre-trained model being modified [1]. This grounds each memory block in meaningful pretrained representations from the outset rather than starting from a random initialization, while bypassing the need for an outer meta-learning loop.

4 Related Work

As stated in the introduction, the Nested Learning paper focused on addressing the issues of catastrophic forgetting and long-context reasoning by presenting three contributions: expressive optimizers, a self-modifying sequence model, and the Continuum Memory System. This project focuses exclusively on CMS and its relation to improved continual learning. In accordance with that, this section focuses on the broader landscape of memory-augmented Transformers, where CMS fits in that space, and prior architectural approaches to continual learning.

Memory-Augmented Transformers is a categorical term for any Transformer model that often, but not always, seeks to more closely align with the human brain by incorporating structural changes which address the "fixed-context and static-knowledge constraints of standard model" [2]. In a 2025 paper by researchers at Huawei Technologies, they provided a comprehensive examination of existing techniques and transformers which implemented such changes, creating a taxonomy of existing architectures for Memory-Augmented Transformers across three dimensions: functional objectives, memory representations, and integration mechanisms [2]. Memory-Augmented Transformers get classified across all three dimensions simultaneously. Within each dimension, the paper identified several subcategories, totaling twelve across the full taxonomy. Despite the breadth of this taxonomy, CMS was not among the techniques discussed. Its absence from this comprehensive 2025 survey suggests it has not yet been widely examined as a memory augmentation technique. Using their proposed taxonomy, CMS could be classified as follows: (1) Functional Objective — out-of-distribution learning and adaptation, specifically targeting catastrophic forgetting; (2) Memory Representation — parameter-encoded memory, as knowledge is stored directly in the model's weights via multi-frequency MLP blocks; and (3) Integration Mechanism — gated control, as the multi-frequency update structure regulates which weights update quickly and which update slowly. The idea that CMS could be classified within the taxonomy situates CMS within architectural memory research and motivates the experiment described in Section 6.

CMS is not the first proposal for modifying the Transformer architecture as a means to improve continual learning. While several architectural approaches to continual learning exist, Titans are among the most directly comparable to CMS given its shared motivation and research lineage. The Titans architecture addresses the challenge of catastrophic forgetting through a surprise-driven mechanism to learn while retaining past knowledge [2]. Referred to as a new family of architectures, Titans differ from standard Transformer models in that it includes a dedicated neural long-term memory module alongside standard attention, in which inference events that violate expectations trigger updates to the memory weights [3]. With CMS, Google researchers proposed replacing the standard FFN block entirely with a chain of multi-frequency MLP blocks, embedding the memory mechanism directly into the model's core structure rather than augmenting it with additional components [1].

5 Method

5.1 Dataset

The dataset used for this project was CLINC150, a classification dataset for intent detection in natural language utterances [4]. CLINC150 contains 150 intent classes organized across 10 domains: *banking*, *credit_cards*, *kitchen_and_dining*, *home*, *auto_and_commute*, *travel*, *utility*, *work*, *small_talk*, and *meta*. Each domain contains 15 intents across the 150 in-scope classes. This dataset was selected in part because it is one of the datasets used by the Google researchers in their continual learning experiments for the Nested Learning paper, supporting closer comparability with their findings [1].

The dataset includes an additional "out-of-scope" (OOS) class representing text that does not belong to any of the 150 intents. Since this project focuses on evaluating forgetting across known intent classes, all OOS examples were removed prior to any training or evaluation. Following OOS removal, the intent label indices contained a gap at the position previously occupied by the OOS class. The labels were therefore remapped to a continuous range of 0 to 149 to ensure compatibility with the model's output layer. A domain column was added to each datapoint by mapping each intent name to its corresponding domain using the intent-to-domain dictionary provided in the official CLINC150 repository [4]. This domain assignment was essential for organizing the sequential training and evaluation procedure described in Section 5.3.

All text inputs were tokenized using the DistilBERT tokenizer with a maximum sequence length of 40 tokens, padding to maximum length, and truncation enabled. A maximum length of 40 was chosen based on the observed maximum token length across all splits, providing a small buffer while keeping sequences compact. The tokenized dataset was then organized into per-domain `DataLoader` objects with a batch size of 32, producing separate train, validation, and test loaders for each of the 10 domains.

5.2 Model

5.2.1 Base Model

The Google Nested Learning paper used LLaMA-3-8B and LLaMA-3 as base models [1]. For this experiment, a more lightweight encoder-only model was chosen: DistilBERT. An encoder-only model was chosen due to the CLINIC150 dataset being classification based. The particular model selection was informed by a comparative study evaluating three transformer architectures on the CLINC150 dataset, which found DistilBERT to achieve accuracy comparable to the best-performing model while being more efficient [5]. And thus, DistilBERT was chosen for this project for its performance and efficiency.

DistilBERT was loaded as a sequence classification model with 150 output labels corresponding to the 150 in-scope intent classes. DistilBert is composed of an embedding module, six layers of transformer blocks, pre-classification linear layer, an added classification linear head, and a dropout layer. The underlying structure which is most relevant to this project is that of the transformer block and FFN, see Figure 1.

5.2.2 Architectural Adjustment: Replacing FFN with CMS Block

The core architectural contribution of this work was replacing the FFN blocks in each of DistilBERT's six transformer layers with a CMS, see figure 2. As detailed in section 3.1, a CMS block is a chain of MLP sub-blocks operating at different update frequencies, designed to preserve knowledge from earlier tasks while allowing adaptation to new ones. Note that while MLP is the term formally

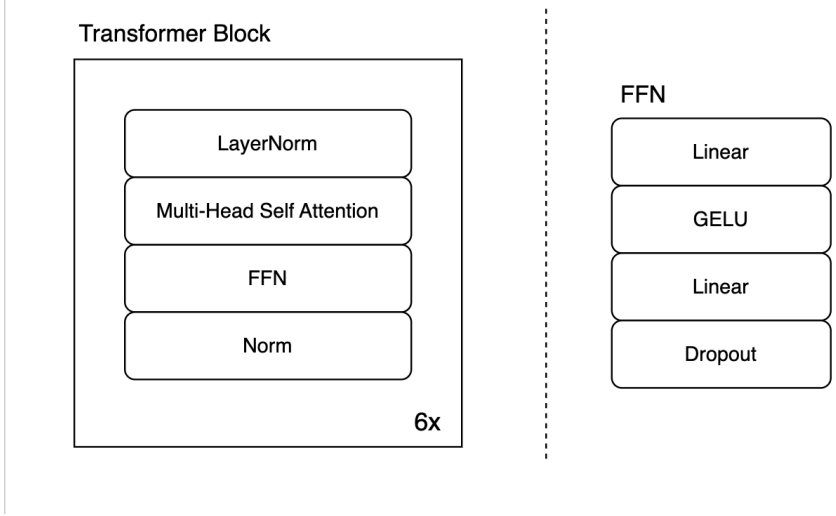


Figure 1: DistilBert Transformer Architecture

used when talking about CMS, in practice this is equivalent to FFN.

The implementation details for the FFNs were designed to mirror DistilBert's structure: the linear layers mapped from 768 to 3072 and 3072 to 768, and GELU activation function was used. The CMS block implemented in this project for experiments (see Section 6) was composed of three FFN layers. Each FFN sublayer was initialized from DistilBERT's pretrained FFN weights rather than random initialization, providing a stable starting point grounded in pretrained representations. Additionally, for this project the Sequential CMS variant was selected for implementation due to its straightforward forward pass structure and compatibility with the ad-hoc pretrained initialization strategy, which bypasses the meta-learning requirement (see Section 3.1.1).

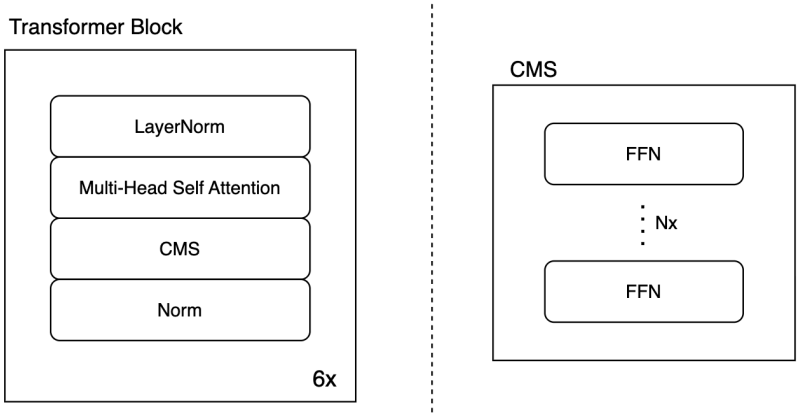


Figure 2: Modified DistilBert Transformer Architecture with CMS Layer

5.3 Training and Testing

Training followed a sequential, domain-by-domain continual learning protocol. The 10 domains were presented to the model in alphabetical order. At each step, the model was trained exclusively on the training data for the current domain. The classification target was the intent label (one of 150 classes). After completing training on domain i, the model was evaluated on the test sets of all domains seen so far $(0, \ldots, i)$.

As is common practice in typical transformer training, when training the base model in the continual learning experiment a single optimizer was used. This differed from that of training the modified model in that each MLP layer (total three) had its own optimizer that was shared across transformer layers. The gradient got accumulated for $C^{(\ell)}$-1 steps and then applied to update the weight parameters via stepping the optimizer, equation 4 in Section 3.1. There was a single shared optimizer used for all non-CMS parameters (attention, embeddings, classifier head, etc.) that followed standard practice of updating every step during training. Both the base model and modified model used AdamW as the optimizer.

5.4 Evaluation Metrics

Four complementary metrics were used to assess and compare model performance from the continual learning experiment.

Per-Domain Accuracy Matrix. From the training and testing data, an accuracy matrix was visualized as a heatmap, providing an interpretable view of both learning and forgetting dynamics across all domains and training stages. The diagonal entries $R_{i,i}$ represent performance on domain i immediately after training on it, and off-diagonal entries $R_{i,j}$ (where $j < i$) capture how the model performed on previously learned domains after subsequent training stages.

Average Accuracy (ACC). ACC measures overall retention after all tasks have been learned:

$$ACC = \frac{1}{T} \sum_{i=1}^{T} R_{T,i} \tag{5}$$

where T is the total number of tasks (domains) and $R_{T,i}$ is the accuracy on task i after training on all T tasks [6].

Backward Transfer (BWT). BWT quantifies the average change in performance on earlier tasks caused by subsequent training:

$$BWT = \frac{1}{T-1} \sum_{i=1}^{T-1} (R_{T,i} - R_{i,i}) \tag{6}$$

where T is the total number of tasks (domains), $R_{T,i}$ is the accuracy on task i after training on all T tasks, and $R_{i,i}$ is the accuracy on task i immediately after training on task i [6]. A negative BWT indicates forgetting, while a value near zero indicates the model retained prior knowledge. A positive BWT would indicate that training on later tasks improved performance on earlier ones.

Average Diagonal Average (ADA). While ACC and BWT evaluate retention after the full training sequence, neither captures how well the model learned each domain at the time it was trained. ADA addresses this by averaging the diagonal entries of the accuracy matrix:

$$ADA = \frac{1}{T} \sum_{i=1}^{T} R_{i,i} \tag{7}$$

where $R_{i,i}$ is the accuracy on task i immediately after training on task i. This metric was introduced to evaluate whether the CMS architecture impacted the model's ability to learn each domain directly after training, independent of retention.

6 Experiment

A continual learning experiment was run on the base DistilBERT model and several hyperparameter variations of the modified model, hereon referred to as DistilBERT-CMS. The purpose was to evaluate how each model performed in retaining knowledge of previously learned tasks across sequential domain training.

The base model performed well on the domain it was currently trained on but exhibited clear catastrophic forgetting on already-learned tasks. This is visible in the per-domain accuracy matrix of the base model (see Figure 3a), where early domains lose accuracy sharply as training continues on later domains. This forgetting is further confirmed by the calculated BWT, ACC and ADA scores (see row id 0 in Table 1): the model learns each domain well in isolation, evidenced by a strong ADA, but fails to retain that knowledge once training moves on.

Several experiments were then conducted on DistilBERT-CMS. By default, the CMS architecture replaced each FFN block in DistilBERT's six transformer layers with a chain of 3 MLP blocks governed by chunk sizes $C^{(\ell)} = [1, 2, 4]$, where smaller chunk sizes correspond to higher update frequency (short-term memory) and larger chunk sizes to lower update frequency (long-term memory). Each experiment varied one or more hyperparameters to isolate the effect of specific design choices:

1. Same hyperparameters as base model — does CMS alone reduce catastrophic forgetting?
2. Shared step counter across domains — does step continuity across domain boundaries improve results as it would impact update frequency?
3. Increased epochs — does CMS need more training time to outperform the base model and reach loss convergence?
4. Increased epochs + shared step counter — does step continuity compound with longer training?
5. Increased epochs + lower learning rate — does a smaller learning rate reduce overwriting of previously learned parameters?
6. Increased epochs + wider C^{ℓ} sizes — does a larger frequency spread improve long-term retention?
7. Dynamic epochs (early stopping per domain) — does adapting training length per domain yield better results than a fixed epoch count?

The full results of these experiments are shown in Table 1. While catastrophic forgetting remains evident across all experiments, every DistilBERT-CMS configuration produced a less negative BWT score than the base model, indicating that the CMS architecture consistently reduced, though did not eliminate, forgetting. It should be noted that the BWT improvements seen in experiments corresponding to row ids 1, 2, 5, and 6 in Table 1 are not enough to be indicative of better performance as all four configurations produced an ACC of approximately 0.66%, suggesting the models failed to meaningfully learn across the task sequence. In this context, a less negative BWT is not a reliable indicator of better retention — a model that never learns much has less to forget. These configurations are therefore considered underperforming and their BWT gains are treated as negligible.

The more informative comparisons come from experiments seen in row ids 3, 4, and 7 of Table 1, where models achieved more meaningful ACC scores, while still being lower than the base model. The best-performing configuration was DistilBERT-CMS trained for 25 epochs (row id 3 in Table 1), which improved BWT from -95.43 percentage points (pp) to -64.15 pp relative to the base

model. Its pre-domain accuracy matrix can be seen in Figure 3b. Notably, the dynamic epoch experiment (row id 7 in Table 1) achieved an ACC of 9.18%, close to the base model's 10.00%, making it the CMS configuration most comparable to the base model in terms of overall learning. However, despite this stronger learning signal, its BWT improvement was smaller than that of the fixed 25-epoch configuration, suggesting that adapting training length per domain did not translate into better long-term retention (see Figure 3c for pre-domain accuracy matrix). One notable finding across all CMS experiments was that all ADA scores were lower than the base model. This suggests a potential tradeoff inherent to the CMS design: by distributing parameter updates across blocks with varying frequencies, the model sacrifices some immediate learning capacity in exchange for better retention of prior knowledge.

	Experiment	ADA	BWT	ACC
0	Base - 10 epochs, lr 2e-5, chunk_sizes=[1,2,4]	95.8889	-95.4321	10.0000
1	CMS - same hyperparams as base	19.5111	-20.9383	0.6667
2	CMS - shared step counter	8.5556	-8.7654	0.6667
3	CMS - 25 epochs	62.6222	-64.1481	4.8889
4	CMS - 25 epochs, shared step count	66.4222	-64.9383	7.9778
5	CMS - 25 epochs, lr 5e-6	72.9778	-80.3457	0.6667
6	CMS - 25 epochs, chunk_size=[1,4,16]	31.6222	-34.3951	0.6667
7	CMS - dynamic epochs	89.5778	-89.3333	9.1778

Table 1: All Experiment Evaluation Results

7 Conclusion

Inspired by the findings detailed in Google's 2025 paper "Nested Learning: The Illusion of Deep Learning Architecture", this project isolated one of three of their contributions to enhancing continual learning and long-context reasoning. This project focused on investigating the effectiveness of Continuum Memory System (CMS) blocks, a multi-frequency stacked MLP design, as an architectural modification to reduce catastrophic forgetting seen in continual learning for transformer models. By replacing the standard FFN layers in DistilBERT, an encoder-only classification model, with CMS blocks, this study evaluated whether frequency-controlled parameter updates could reduce catastrophic forgetting in a sequential domain learning setting. The results demonstrated that CMS consistently reduced catastrophic forgetting compared to the baseline model, as evidenced by less negative Backward Transfer scores across all experimental configurations. This supports the hypothesis that distributing parameter updates across multiple temporal frequencies can improve long-term knowledge retention. However, the improvement seen was minimal and, while reduced, catastrophic forgetting was still present. Additionally, the improvement in long-term memory came with the side effect in which the CMS-based models exhibited reduced learning capacity on current tasks, reflected in lower accuracy scores. This suggests that while slower update frequencies help preserve previously learned knowledge, they limit the model's ability to rapidly adapt to new information - which was also evident by the CMS variant of DistilBert needing more epochs during training to reach loss convergence.

Overall, this study positioned CMS as a promising architectural direction for continual learning in transformer models, particularly for applications where long-term knowledge retention is critical. That being said, the results of this study heavily imply that architectural adjustments of the transformer model's FFN to CMS is not sufficient enough to produce meaningful change in catastrophic forgetting to warrant its reduced ability on short-term learning. Future work of this project would would benefit from further investigation into the two other ideas Google presented - (1) more expressive optimizers, and (2) a self-modifying sequence model that learns its own update algorithm - and evaluate to what degree those implementations along with CMS would improve performance [1]. Additionally, it would be valuable to extend CMS implementation into larger-scale generative models and conduct more complex tasks as a means to provide further insight into scalability and practical impact.

References

[1] A. Behrouz, M. Razaviyayn, P. Zhong, and V. Mirrokni, "Nested Learning: The Illusion of Deep Learning Architectures," arXiv:2512.24695, Dec. 2025.

[2] P. Omidi, X. Huang, A. Laborieux, B. Nikpour, T. Shi, and A. Eshaghi, "Memory-Augmented Transformers: A Systematic Review from Neuroscience Principles to Enhanced Model Architectures," arXiv preprint arXiv:2508.10824, 2025.

[3] A. Behrouz, P. Zhong, and V. Mirrokni, "Titans: Learning to Memorize at Test Time," arXiv preprint arXiv:2501.00663, 2024.

[4] S. Larson, A. Mahendran, J. J. Peper, C. Clarke, A. Lee, P. Hill, J. K. Kummerfeld, K. Leach, M. A. Laurenzano, L. Tang, and J. Mars, "An Evaluation Dataset for Intent Classification and Out-of-Scope Prediction," in *Proc. 2019 Conf. Empirical Methods in Natural Language Processing and 9th Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP)*, 2019.

[5] V. S. Solanki, E. Sudhanshu Sharma, L. Sindhu, L. Bisht, and H. Singh, "BERT, RoBERTa, and DistilBERT for Intent Classification: A CLINC-150 Evaluation with QA Applications," in *Proc. 2025 Int. Conf. Computing Technologies (ICOCT)*, 2025, pp. 1–6. doi: 10.1109/ICOCT64433.2025.11118772

[6] D. Lopez-Paz and M. Ranzato, "Gradient Episodic Memory for Continual Learning," arXiv preprint arXiv:1706.08840, 2017.

Continual Learning Accuracy Matrix (%)

trained_on \ evaluated_on	auto_and_commute	banking	credit_cards	home	kitchen_and_dining	meta	small_talk	travel	utility	work
auto_and_commute	98.000	-	-	-	-	-	-	-	-	-
banking	0.000	94.667	-	-	-	-	-	-	-	-
credit_cards	0.000	0.000	94.444	-	-	-	-	-	-	-
home	0.000	0.000	0.000	94.444	-	-	-	-	-	-
kitchen_and_dining	0.000	0.000	0.000	0.000	92.667	-	-	-	-	-
meta	0.000	0.000	0.000	0.000	0.000	96.222	-	-	-	-
small_talk	0.000	0.000	0.000	0.000	0.000	0.000	96.444	-	-	-
travel	0.000	0.000	0.000	0.000	0.000	0.000	1.556	98.222	-	-
utility	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	97.556	-
work	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	3.778	96.222

(a) Base DistilBERT

Continual Learning Accuracy Matrix (%)

trained_on \ evaluated_on	auto_and_commute	banking	credit_cards	home	kitchen_and_dining	meta	small_talk	travel	utility	work
auto_and_commute	90.667	-	-	-	-	-	-	-	-	-
banking	2.444	6.667	-	-	-	-	-	-	-	-
credit_cards	0.000	0.000	87.111	-	-	-	-	-	-	-
home	0.000	0.000	10.889	62.444	-	-	-	-	-	-
kitchen_and_dining	0.000	0.000	0.000	0.000	10.889	-	-	-	-	-
meta	0.000	0.000	0.000	0.000	0.000	82.889	-	-	-	-
small_talk	0.000	0.000	0.000	0.000	0.000	13.556	52.222	-	-	-
travel	0.000	0.000	0.000	0.000	0.000	0.000	1.556	89.778	-	-
utility	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.222	94.667	-
work	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	48.889

(b) DistilBERT-CMS, 25 Fixed Epochs

Continual Learning Accuracy Matrix (%)

trained_on \ evaluated_on	auto_and_commute	banking	credit_cards	home	kitchen_and_dining	meta	small_talk	travel	utility	work
auto_and_commute	92.889	-	-	-	-	-	-	-	-	-
banking	0.000	84.667	-	-	-	-	-	-	-	-
credit_cards	0.000	1.556	85.333	-	-	-	-	-	-	-
home	0.000	0.000	0.222	79.556	-	-	-	-	-	-
kitchen_and_dining	0.000	0.000	0.000	0.889	88.000	-	-	-	-	-
meta	0.000	0.000	0.000	0.000	0.000	90.444	-	-	-	-
small_talk	0.000	0.000	0.000	0.000	0.000	4.667	92.889	-	-	-
travel	0.000	0.000	0.000	0.000	0.000	0.000	0.000	94.000	-	-
utility	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	96.222	-
work	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	91.778

(c) DistilBERT-CMS, Dynamic Epochs

Figure 3: Per-Domain Accuracy Matrices for the base model and two notable CMS configurations. This is displayed as a heat map with darker blue indicating higher accuracy and red indicating forgetting.